Lecture 6: Data Manipulation and Analysis (II) – Base R

Harris Coding Camp – Standard Track

Summer 2024

Today's Lecture

We will cover:

- `$ + <-` to create new columns
- `summary()`, `aggregate()` to summarize data
- Introduction to linear relationship between two variables

Creating new columns using operators

Creating new columns

storm	wind	pressure	date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1999-06-13
Arthur	45	1010	1996-06-21

storm	wind	pressure	date	ratio	inverse
Alberto	110	1007	2000-08-12	9.15	0.11
Alex	45	1009	1998-07-30	22.42	0.04
Allison	65	1005	1995-06-04	15.46	0.06
Ana	40	1013	1997-07-01	25.32	0.04
Arlene	50	1010	1999-06-13	20.20	0.05
Arthur	45	1010	1996-06-21	22.44	0.04

We may want to create new columns such as

- ratio $=$ pressure$/$wind
- inverse $= 1/$ratio

Creating new columns using $ and <-

We can add new columns to a data frame using $ and <-

Syntax: `data_name$new_var <- <definition>`

- `data_name`: name of the data frame
- `new_var`: name of the new column
- `<definition>`: how the new column is defined

```
# create new column 'ratio'
# volume / listings
housing_data$ratio <-
  housing_data$volume / housing_data$listings
```

Creating new columns using $ and <-

If we want to create new column with **multiple** levels:

- First, create new column `r_level` with 0
- If `ratio` is at least 10000, assign `H` to `r_level`
- If `ratio` is between 8000 and 10000, assign `M` to `r_level`
- If `ratio` is less than 8000, assign `L` to `r_level`

Syntax: `data_name$new_var[condition] <- new_value`

```
# create a new column r_level
housing_data$r_level <- 0
housing_data$r_level[housing_data$ratio >= 10000] <- "H"
housing_data$r_level[housing_data$ratio >= 8000 &
                     housing_data$ratio < 10000] <- "M"
housing_data$r_level[housing_data$ratio < 8000] <- "L"
```

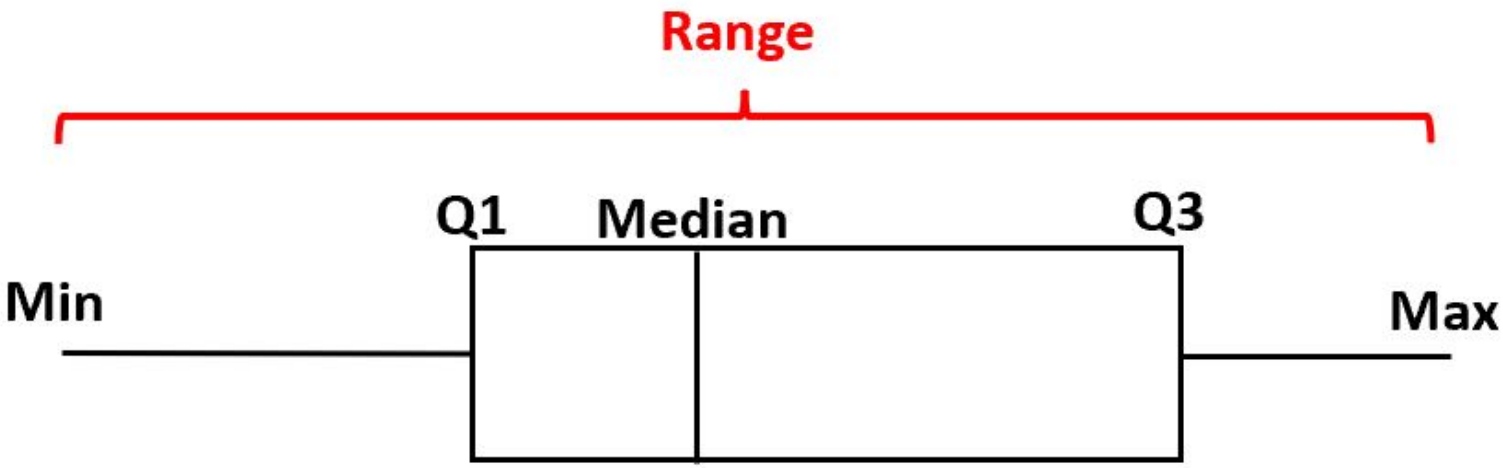
We can also use `case_when()` in this case. Stay tuned!

Try it yourself!

1. Using `housing_data`, select the following variables (city, year, month, sales, volume) and create a new column called `mean_price`, defined as a ratio of volume to sales. Include this newly created variable and all variables from the original data frame to a new one called `housing_data_q1`.
2. Use the code from part 1 to select rows with `mean_price` at least $70,000.
3. Create a **binary indicator variable** called `ref` that identifies whether the mean price is too high (i.e. `ref` equals 1 if `mean_price` is at least $100,000; 0 otherwise).

Summarizing data

Summarizing data using `max()`, `min()`, `range()`



- Range = Max - Min
- Q1: 1st quartile (25%)
- Median: 2nd quartile (50%)
- Q3: 3rd quartile (75%)

Summarizing data using `max()`, `min()`, `range()`

Next, we want to find out the max, min and range of `sales`.

Syntax: `function(data_name$var_name)`

```
max(housing_data$sales, na.rm = TRUE) # remove NA
min(housing_data$sales, na.rm = TRUE)
range(housing_data$sales, na.rm = TRUE)

# What if NAs are not excluded?
max(housing_data$sales)
min(housing_data$sales)
range(housing_data$sales)
```

Summarizing data using `median()`, `quantile()`

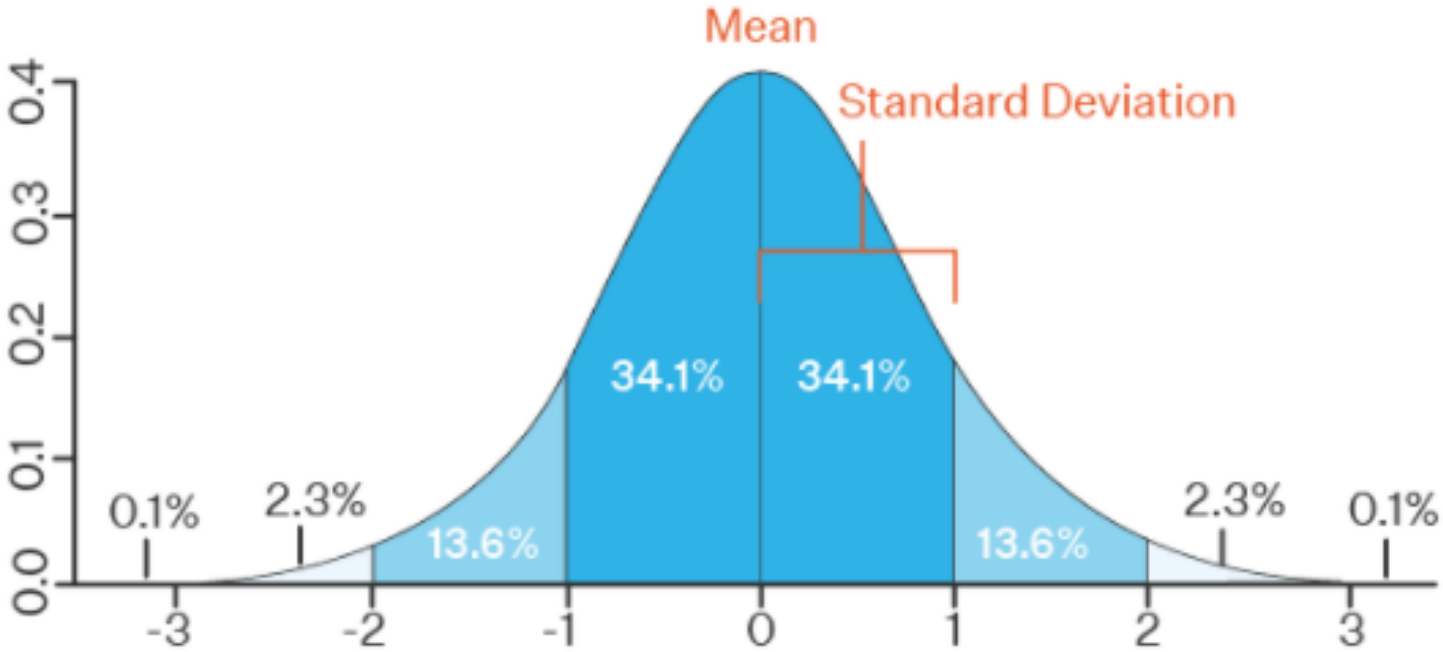
Next, we want to find out the Q1, median and Q3 of `sales`.

Syntax: `function(data_name$var_name)`

```
median(housing_data$sales, na.rm = TRUE) # remove NA
quantile(housing_data$sales, na.rm = TRUE)

# What if NAs are not excluded?
median(housing_data$sales)
quantile(housing_data$sales)
```

Summarizing data using `mean()`, `var()`, `sd()`



- Mean: average of a set of numbers
- Variance: a measure of how data points differ from the mean
- Standard deviation: square root of variance

Summarizing data using `mean()`

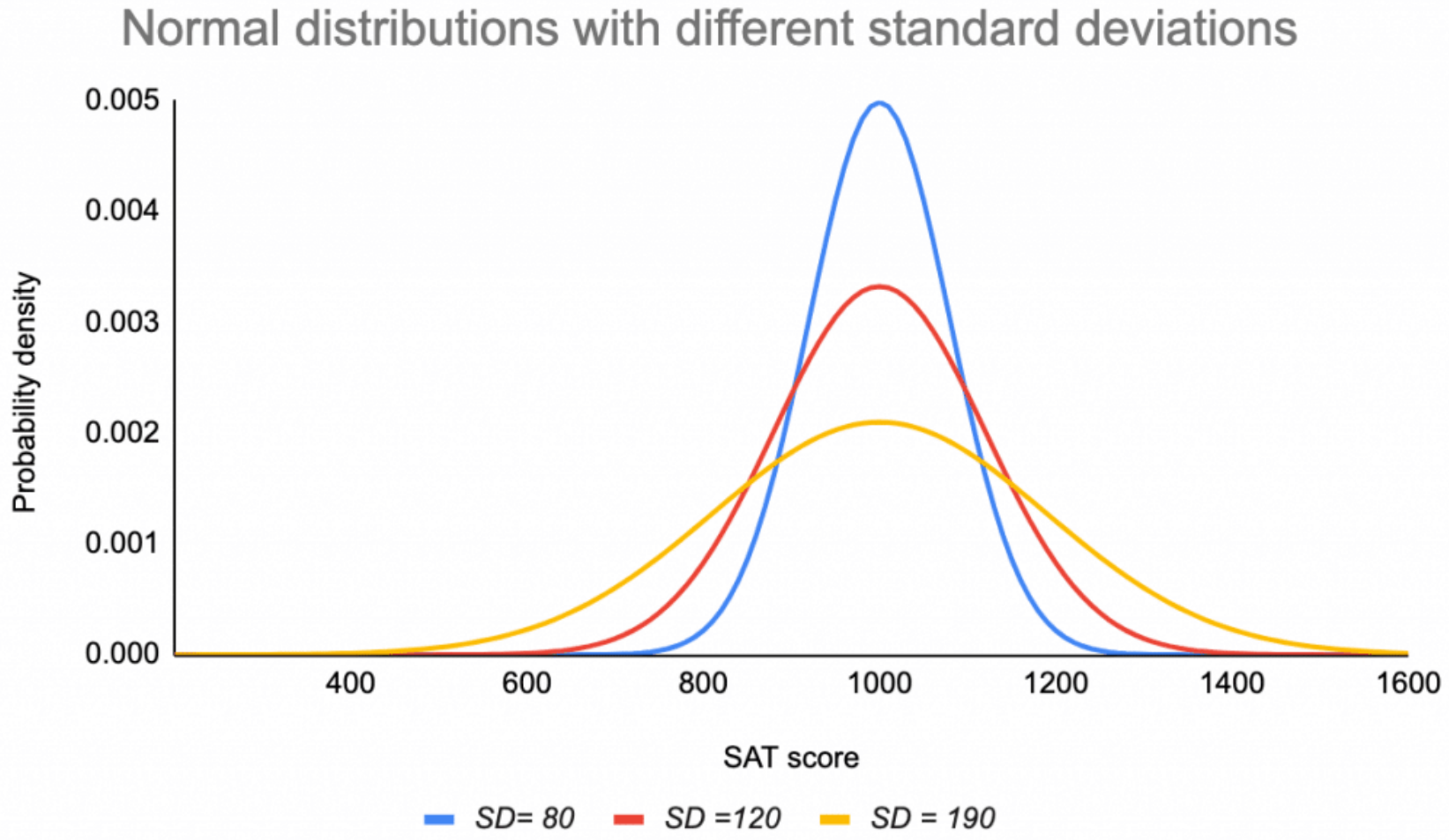
Often, it is useful to learn how the data "looks like" by knowing some basic statistics on the entire dataset. Suppose we want to calculate the mean of `sales`.

Syntax: `mean(data_name$var_name)`

```
mean(housing_data$sales)
mean(housing_data$sales, na.rm = TRUE) # remove NA
```

- If there is at least one missing value in the dataset, use `na.rm = TRUE` to exclude the missing values

Summarizing data using `var()`, `sd()`



Variance (and hence **Standard deviation**) is a measure of how far a set of data are spread out from their mean value

- the more widely spread the data, the larger var (and hence sd)

Summarizing data using `var()`, `sd()`

Let's find out the variance and standard deviation of `sales`.

Syntax: `var(data_name$var_name)`, `sd(data_name$var_name)`

```
var(housing_data$sales, na.rm = TRUE)
sd(housing_data$sales, na.rm = TRUE)

# What if NAs are not excluded?
var(housing_data$sales)
sd(housing_data$sales)
```

Summarizing data using `summary()`

We can compute the minimum, Q1, median, Q3 and maximum for all **numeric** variables of a dataset at once using `summary()`.

Syntax: `summary(data_name)`

```
summary(housing_data)
```

Summarizing data using `summary()`

Imagine we want to obtain the summary info on `sales`, separately for Amarillo and Austin:

```
# all observations
summary(housing_data$sales)
# all Amarillo observations
summary(housing_data$sales[housing_data$city == "Amarillo"])
# all Austin observations
summary(housing_data$sales[housing_data$city == "Austin"])
```

Summarizing data using aggregate()

`aggregate()` allows to split the data into **subgroups** and then to compute summary statistics for each subgroup.

Syntax: `aggregate(y ~ x, data_name, function)`

- `y ~ x`: `y` is numeric variable to be split into groups according to `x`
- `data_name`: name of the data frame
- `function`: a function to compute the summary statistics which can be applied to all data subgroups

```
# Find mean sales by cities
aggregate(sales ~ city, housing_data, mean)
```

```
# Find sd of sales by year and cities
aggregate(sales ~ year + city, housing_data, sd)
```

Try it yourself!

Use `housing_data` and `mean_price` to complete the tasks:

1. Find the maximum, minimum and range of `mean_price` on the entire dataset.
2. Find the mean, variance and standard deviation of `mean_price` on the entire dataset.
3. Redo part 2 for each city, each year. (Hint: use `aggregate()` to do this efficiently!)
4. Create a new variable called `scaled_mean_price`, defined as `mean_price` multiplied by 10.

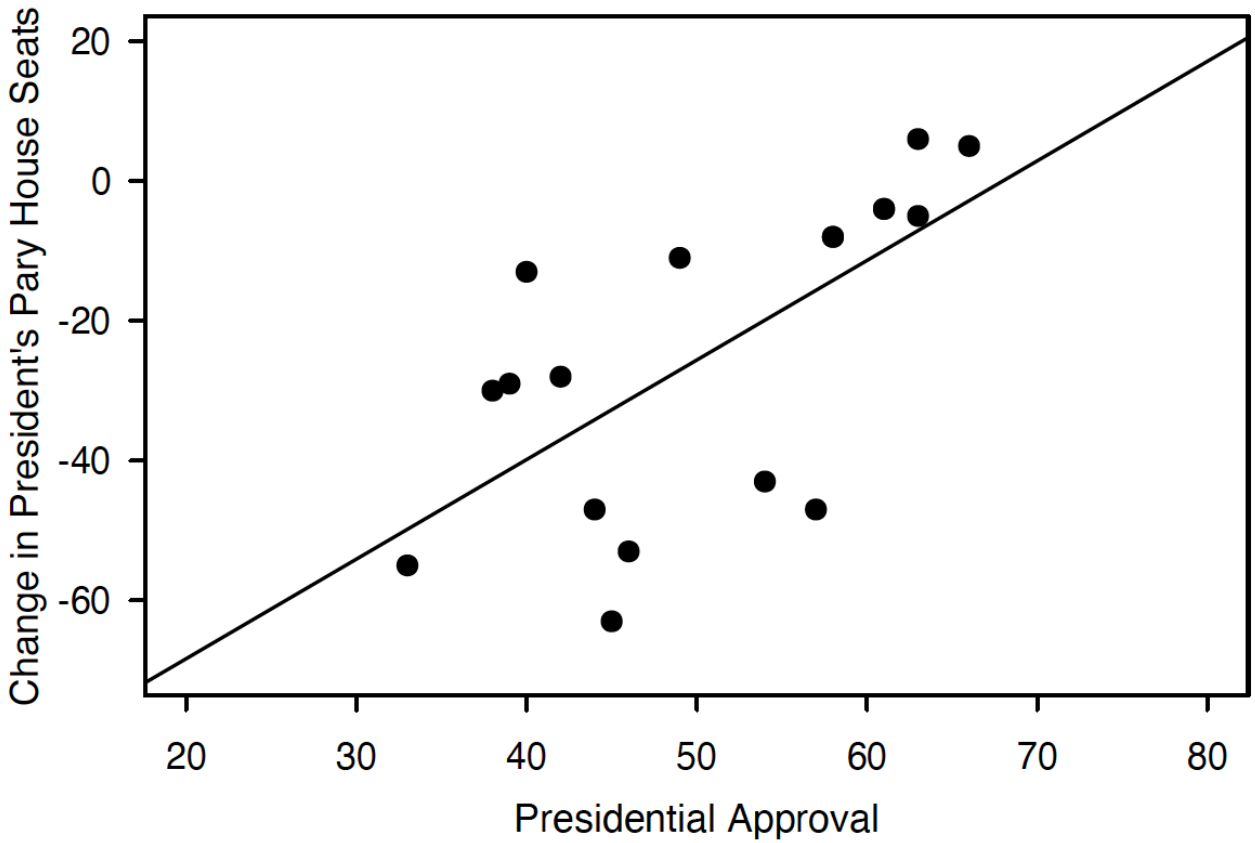
- Find the range of `scaled_mean_price` on the entire dataset. Compare the your finding with part 1.
- Find the mean, var and sd of `scaled_mean_price` on the entire dataset. Compare the your findings with part 2.

Linear relationship and regression model

Linear relationship between two variables

Description and prediction is broadly useful across different fields.

- Can the presidential approval rate tell us sth about the result of midterm election?



Presidential popularity and the midterm results

- How does the popularity of a president predict how well their party will do in the midterm elections?
- Dataset with information on approval and midterm election outcomes:

Variable	Description
`year`	Midterm election year
`president`	Name of president
`party`	Democrat or Republican
`approval`	Gallup approval rating at midterms
`seat.change`	Change in the number of House seat's for the president's party

Loading the data

```
midterms <- read.csv("midterms.csv")
head(midterms)
#>   year  president party approval seat.change rdi.change
#> 1 1946     Truman     D       33         -55         NA
#> 2 1950     Truman     D       39         -29        8.2
#> 3 1954 Eisenhower     R       61          -4        1.0
#> 4 1958 Eisenhower     R       57         -47        1.1
#> 5 1962    Kennedy     D       61          -4        5.0
#> 6 1966    Johnson     D       44         -47        5.3
```

Linear regression

- **Prediction**: for any value of x, what's the best guess about y?
- Simplest possible way to relate two variables: a **line**.

$$y = \alpha + \beta x$$

- Syntax: `reg <- lm(y ~ x, data = data_name)`
- Run the regression with `seat.change` as y and `approval` as x:

```
appseats <- lm(seat.change ~ approval, data = midterms)
appseats
#>
#> Call:
#> lm(formula = seat.change ~ approval, data = midterms)
#>
#> Coefficients:
#> (Intercept)     approval
#>     -96.845        1.424
```

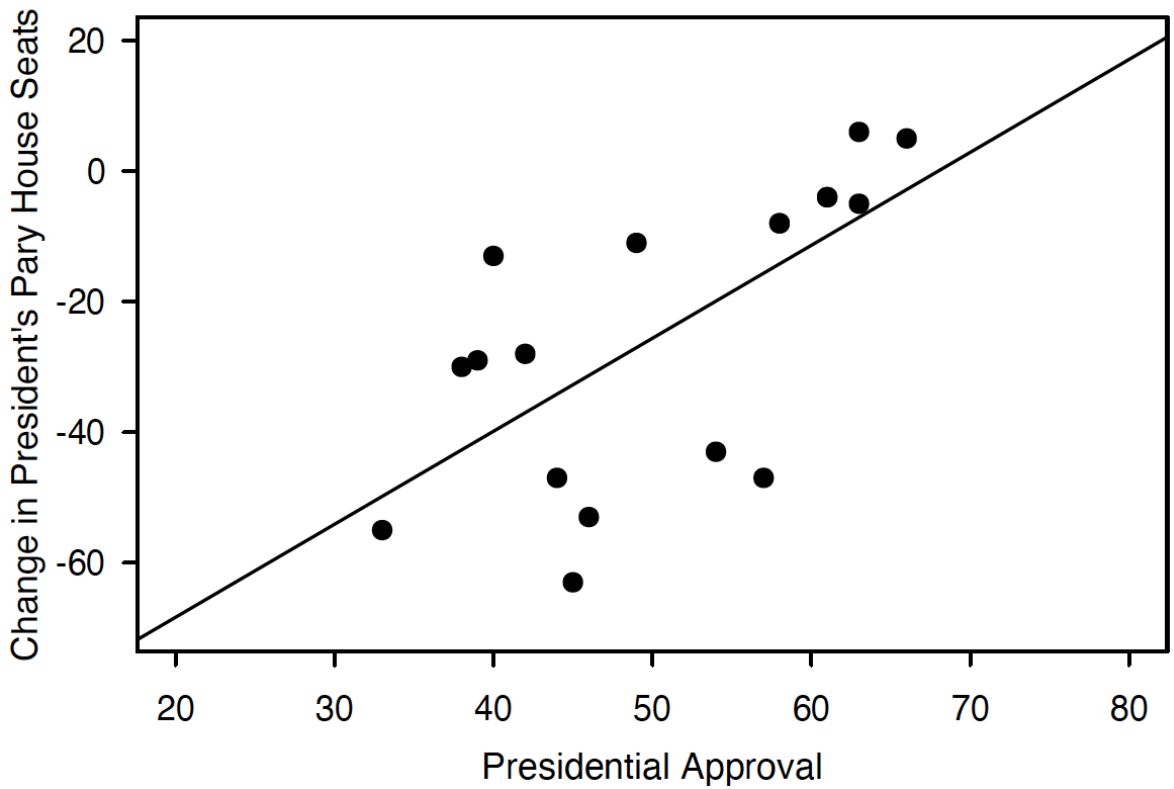
- Intercept: the expected change in House seats when approval is 0
- Slope: the expected change in House seats per unit change in approval

Linear regression

We can impose the **linear regression line** on the scatterplot (next week):

```
plot(midterms$approval, midterms$seat.change,
     xlim = c(20, 80), ylim = c(-70, 20), pch = 19,
     xlab = "Presidential Approval",
     ylab = "Change in President's Pary House Seats")

abline(appseats)
```



Linear regression as prediction

- We don't have the seat change for 2018 entered into our data set.
- What does our model "predict" happened in 2018?

```
tail(midterms)
#>    year president party approval seat.change rdi.change
#> 14 1998   Clinton     D       66           5         5.6
#> 15 2002   W. Bush     R       63           6         2.6
#> 16 2006   W. Bush     R       38         -30         5.7
#> 17 2010     Obama     D       45         -63         3.5
#> 18 2014     Obama     D       40         -13         4.6
#> 19 2018     Trump     R       38          NA         4.1
```

- Use the coefficients we got to create a prediction:
  - $\hat{y}_{2018} = (-96.85)x_{2018} + 1.42 \approx -43$
  - pretty good prediction: Republicans lost 41 seats in 2018!

Recap

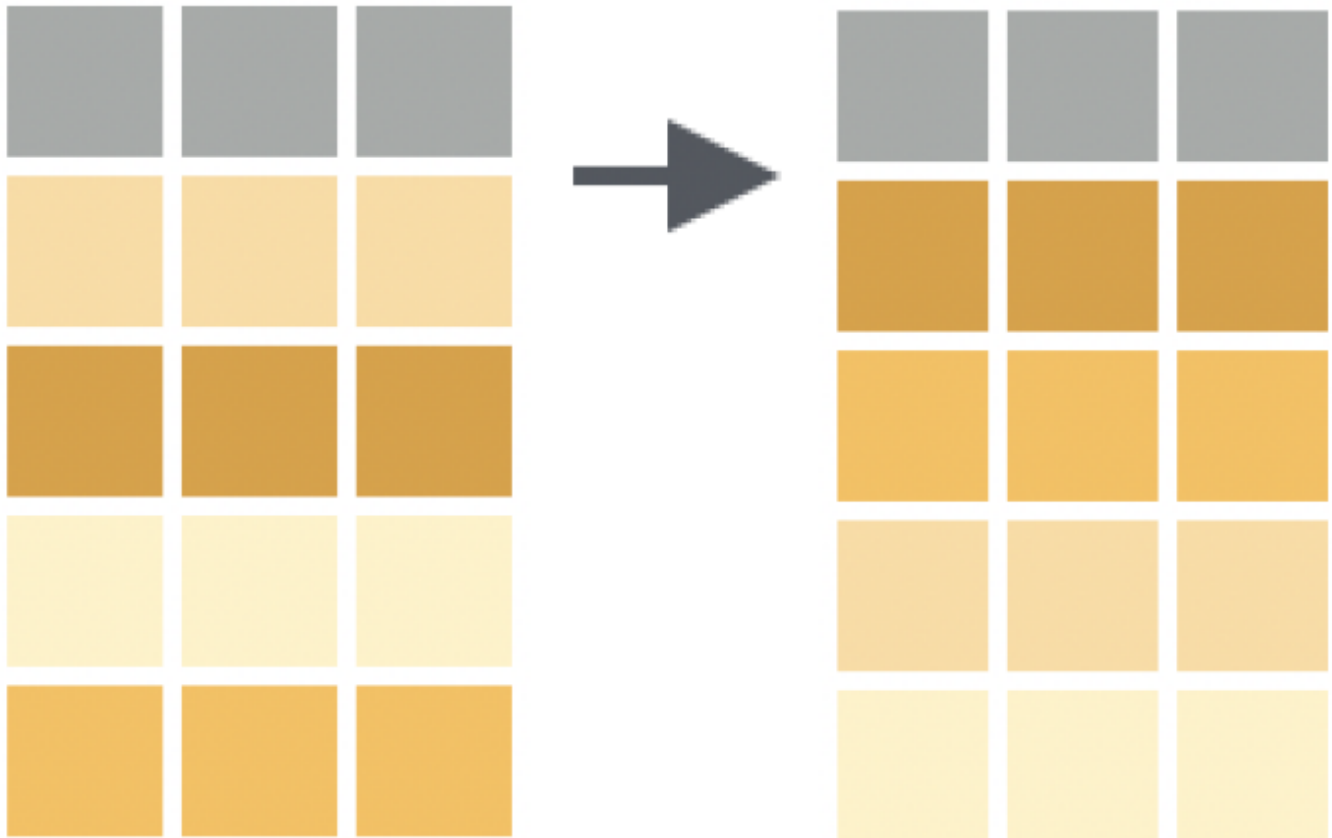
We have covered many useful skills on data manipulation, including:

- `$ + <-` to create new column(s)
- `mean()`, `sd()`, `summary()`, `aggregate()` etc. to summarize data and get a quick idea of how the data "looks like"
- `lm()` to estimate the linear relationship between two variables

Appendix

Sorting data with `sort()` or `order()`

Sorting data with `sort()` or `order()`



Sorting function `sort()`

`sort()` is a Base R function which sorts **vectors**.

Syntax: `sort(x, decreasing = FALSE, ...)`

- `x` is the vector being sorted
- By default, it sorts in ascending order
- To sort in descending order, set the `decreasing` argument to `TRUE`

```
# create a vector
y <- c(1, 55, 30, 26, -5)
# sort the vector
sort(y)
#> [1] -5  1 26 30 55
```

```
sort(y, decreasing = TRUE)
#> [1] 55 30 26  1 -5
```

Sorting function `order()`

`order()` is a Base R function which sorts **data frames**

Syntax: `data_name[order(data_name$var_name, decreasing = FALSE), ]`

- By default, it sorts in ascending order
- To sort in descending order, set the `decreasing` argument to `TRUE`
- Or, use `-` to indicate which variable is descending while using the default ascending sorting

```
# sort by year
housing_data[order(housing_data$year), ]

# sort by year and month
housing_data[order(housing_data$year, housing_data$month), ]

# sort descending via argument
housing_data[order(housing_data$year, housing_data$month,
                   decreasing = TRUE), ]

# sort by both ascending and descending variables
housing_data[order(housing_data$year, -housing_data$month), ]
```

Renaming existing columns using `names()`

Renaming existing columns using `names()`

More commonly used approach:

Syntax:

```
names(data_name)[names(data_name) == "old_name"] <- "new_name"
```

- `data_name`: name of the data frame
- `old_name`: original name of the selected column
- `new_name`: new name of the selected column

```
# change the name of "city" to "location"
names(housing_data)[names(housing_data) == "city"]  <- "location"
```

Correlation coefficient

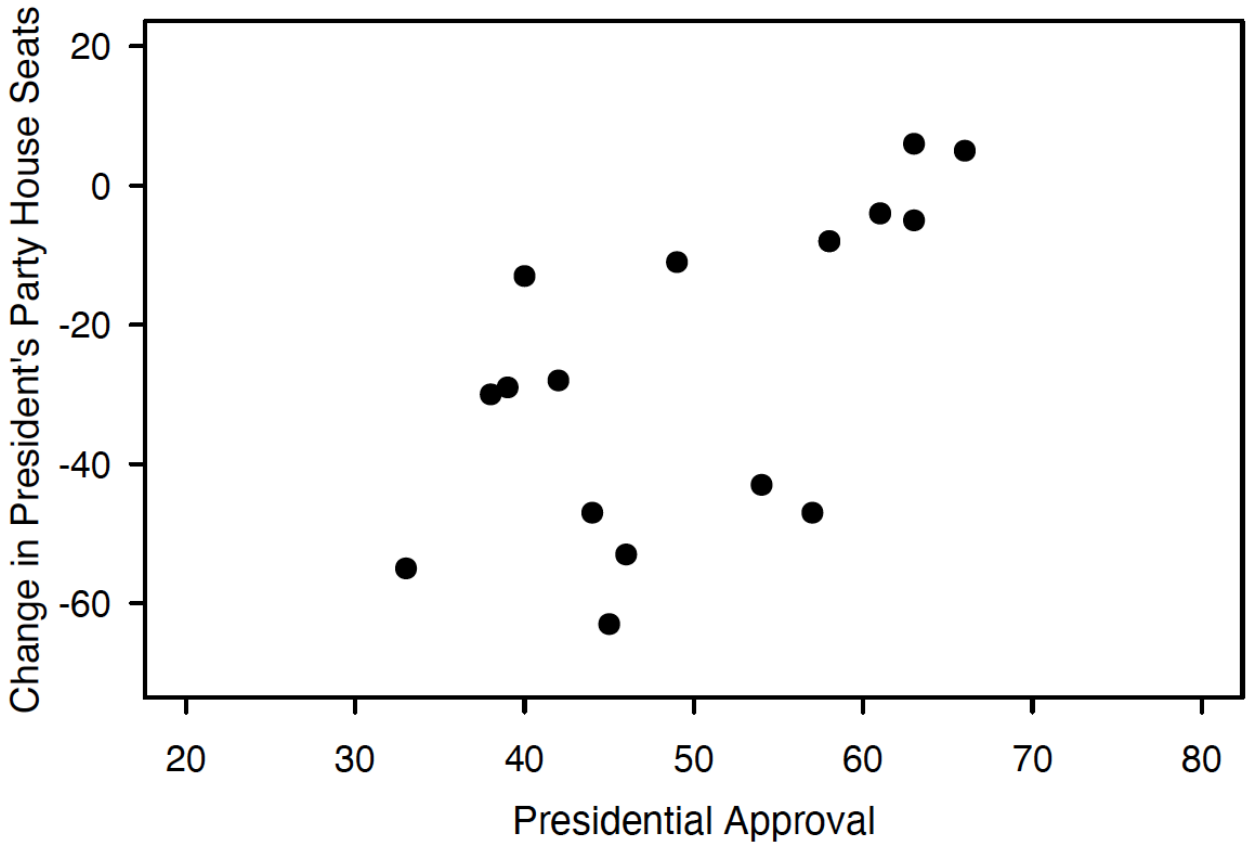
Correlation coefficient

Very often, we are interested in knowing if there is a **linear** relationship (association) between two variables.

- **Correlation coefficient** measures that
- Syntax: `cor(data_name$x, data_name$y)`
- Its values can range from -1 to 1
- If its value (in absolute term) is close to 1, it means stronger linear relationship between `x` and `y`

Correlation coefficient

```
cor(midterms$approval[midterms$year < 2018],
    midterms$seat.change[midterms$year < 2018])
#> [1] 0.6562875
```



- Positive linear relationship between `approval` and `seat.change`